



SEL PLATFORM AND APPLICATION TRANSFER AND HOSTING ARCHITECTURE

Donald Mitchell
Edusightsolutions LLC

TABLE OF CONTENTS

1. Executive overview	SECTION 1 — App Technology & Environment Setup	12. Deployment Architecture & Workflow
2. Business Problem		12.1 Heroku Deployment Process
3. Strategic Objective		12.2 Git-Based Deployment Integration
4. 3 rd Party Hosting Pipelines & Workflows	8. Environment Setup & Development Preparation	12.3 Continuous Integration Considerations
4.1 3 rd Party's GitHub, CLI	8.1 Windows Environment Configuration	
4.2 Bitbucket Pipelines CI/CD, AWS Cloud	8.2 Python Installation & Dependencies	
	8.3 Local Development Setup	
5. Solution Migration & Offboarding Strategy	SECTION 2 — Billing, Heroku Plans, GitHub Plans, & Questions?	SECTION 4 — Heroku Dyno Scaling, and Performance Notes?
5.1 Transition from External Development Team		
5.2 Code Repository Transfer Process	9. Infrastructure & Account Provisioning	13. Application Scaling & Performance Planning
5.3 Application Ownership Transfer	9.1 Heroku Account & Team Setup	13.1 Dyno Sizing Strategy
5.4 Add-on and Service Migration	9.2 GitHub Account & Repository Setup	13.2 Performance Calculations
	9.3 AWS S3 Storage Configuration	13.3 Autoscaling Recommendations
	9.4 Security and Access Considerations	
6. Client Internal Hosting	10. Billing, Licensing, and Cost Considerations	SECTION 5 — GitHub Account Setup & Local Repository Instructions?
6.1 GitHub source code management, collaboration	10.1 Heroku Pricing Models	
6.2 Heroku application hosting using a PaaS model	10.2 GitHub Subscription Plans	
6.3 AWS S3 object storage and supporting data assets	10.3 Cost Optimization Considerations	
7. Architecture Diagram: External to Internal Hosting	SECTION 3 — Offboarding Checklist (GitHub, Heroku, AWS, Add-ons)?	14. Operational Readiness & Sustainability
7.1 3 rd Party Hosting		14.1 Internal Team Roles & Access
7.2 Technical Solution Stack	11. Migration & Offboarding Strategy	14.2 Monitoring & Logging Tools
7.3 Client Internal Hosting	11.1 Transition from External Development Team	14.3 Maintenance and Support Model
	11.2 Code Repository Transfer Process	
	11.3 Application Ownership Transfer	
	11.4 Add-on and Service Migration	

EXECUTIVE OVERVIEW

This Client solution is grounded in concrete technical implementation decisions and centers on three key technical outcomes:

- First, migrating the application and its dependencies from a third-party environment to internal hosting.
- Second, standing up a secure, cloud-based hosting architecture under client ownership.
- Third, defining repeatable deployment and operational workflows.

BUSINESS PROBLEM

From a technical standpoint, the primary issues were cost and external dependency. The application was hosted and managed by a third-party team which was cost prohibitive, particularly during summer, and limited visibility into infrastructure configuration, deployment processes, and performance tuning. This introduced risks in areas such as access control, versioning, and long-term maintainability.

- Client Startup's External dependency
 - Originally, 3rd party service provider maintaining client's Social Emotional Learning (SEL) application targeting k12 school districts.
- Limited control
 - Some frustration felt by client due to slow iteration
- Continuity risks
 - Lost momentum when working with 3rd party.
- Cost Optimization
 - The monthly cost to maintain the platform in that scenario was prohibitive, particularly considering during summer when schools are out and , there's no one using it, and client needed to better utilize funds.

STRATEGIC OBJECTIVE

The objective was to establish internal control across three layers, appropriate to support existing application's language and framework, as well as provide the capability to access logs to perform end-user data analysis and statistics:

- Source code ownership via Git-based version control
- Infrastructure ownership via cloud hosting
- Deployment ownership via defined pipelines and workflows
- Support object storage and data assets

3RD PARTY HOSTING CLIENT APPLICATION (PIPELINES & WORKFLOWS)

[Developer Computer]



▼ (Code Push)

[3rd Party's - GitHub Repository, CLI]



▼ (Automatic Deploy)

[3rd Party Host - Bitbucket Pipelines CI/CD, AWS Cloud)]



└─ Python Web Server (Application Logic)



(Web Request) — [End User Browser]

SOLUTION FOR SMOOTH HAND-OFF FROM 3RD PARTY TO INTERNAL HOSTING OF CLIENT APP

After initial consultation client concluded that they wished to move the application to servers the client controlled:

- ☐ Set up a GH account for an automatic repo transfer
 - ☐ ☐ Set up a GitHub organization.
 - ☐ ☐ Add Client designate e.g. designate@client.com as an admin to a newly created repo or
 - ☐ ☐ Send Client consultant's GH and consultant transfer
 - ☐ ☐ Transfer code
- ☐ Set up a Heroku account for an automatic transfer of the monolith application
 - ☐ ☐ Set up a Heroku team account
 - ☐ ☐ Invite designate@client.com to the team or
 - ☐ ☐ Client can invite consultant as an admin
 - ☐ ☐ Transfer application
- Following add-ons transfer automatically
 - ☐ ☐ Rollbar
 - ☐ ☐ Postgres
 - ☐ ☐ Memcache cloud
 - ☐ ☐ Mailgun
 - ☐ ☐ Papertrail
- Set up an AWS account to use S3 with the appropriate bucket and configuration

CLIENT INTERNAL HOSTING OF APPLICATION (PIPELINES & WORKFLOWS)

[Developer (Internal)]

▼ (Code Push)

[GitHub Repository (Client)]

▼ (Automatic Deploy)

[Heroku Cloud (Client)]

└─ Python Web Server (Application Logic – Internal)

▼ File Uploads / Downloads

[AWS S3 Bucket (Storage – Access Logs), Heroku add-ons (e.g. Postgres, etc.)]

(Web Request) — [End User Browser]

Application Architecture & Technology Stack

- ❑ Supported Languages & Frameworks
- ❑ Hosting Platform Overview (Heroku)
- ❑ Source Code Management (GitHub)

ARCHITECTURE DIAGRAM: EXTERNAL TO INTERNAL HOSTING TRANSITION

3rd party hosting (pipelines & workflows)



External Hosting Environment

- The application was initially hosted on a third-party platform with limited access and external maintenance reliance.

Technical Solution Stack

- GitHub for centralized source code management and collaboration
- Heroku as the application hosting environment using a PaaS model
- AWS S3 for object storage and supporting data assets



Critical Transfer Phase

- Key activities include migrating source code to a client GitHub, transferring app ownership in Heroku, and moving essential add-ons securely.

Internal (pipelines & workflows)



Internal Hosting Architecture

- The final setup features GitHub for source control, Heroku for application hosting, and AWS S3 for scalable storage, enabling full client control.

SECTION 1 — App Technology & Environment Setup

=== Client Application Technology Questions ===

- "What code are you using for your app?"
- Heroku supports: Java, Node.js, Scala, Clojure, Python, and PHP.

=== Windows Environment Setup ===

Operating System:

- Windows 11 Version 23H2

Steps:

1. Install Chocolatey (community system package manager then use to install Python; pushes python 3 as the default).
 - Open and run PowerShell in Administrator mode.
 - Right-click Windows/Start icon; → choose “Terminal (Admin)” or “PowerShell (Admin)” for Administrator mode
 - Run the long Chocolatey installation command (referenced later in notes) to install Chocolatey.
2. Install Python (high level programming language):
 - Command: `choco install python`
 - Python 3 becomes the default.
3. Install pip (Python package installer):
 - Windows Powershell Commands (chelp for Python, pip):
 - `py --version`
 - `py -m pip --version`

SECTION 1 — App Technology & Environment Setup (con't)

=== Client Application Technology Questions ===

- "What code are you using for your app?"
- Heroku supports: Java, Node.js, Scala, Clojure, Python, and PHP.

=== Windows Environment Setup ===

Operating System:

- Windows 11 Version 23H2

Steps (continued):

4. Install Python build tools:

- Instead of setuptools, install the `build` package using pip (This creates new Python packages and/by automatically downloading setup tools and any other build-time dependencies that project might have — Just specify dependencies in a pyproject.toml file.
- This automatically downloads setuptools and other build-time dependencies.
- Dependencies are defined in the pyproject.toml file.
- Command: `pip install --upgrade build`

SECTION 2 — Billing, Heroku Plans, GitHub Plans, and Questions?

=== Heroku Billing & Payment Questions ===

- “Do you want to use your credit card to set up the Heroku account in your or your company’s name?”
- Heroku Eco Dynos (Instances) pricing:
 - Eco Dynos Plan: \$5/month for 1000 dyno hours.
 - Not prorated — always \$5/month regardless of start date.
 - Personal apps can use Eco Dynos (apps not part of a Heroku Team or Enterprise).
- Question: “How do you want to handle payment for any paid services on GitHub or Heroku?”
- All credit card details: [REDACTED]

=== GitHub Plans ===

- GitHub Free: \$0/month per user.
- GitHub Team: \$4/month per user (for individuals and organizations).
- GitHub Enterprise: \$21/month per user.
- Question: “Do you want to upgrade to GitHub Team or GitHub Enterprise Cloud (To get the full feature set on private repositories and additional features at the organization level, including SAML single sign-on and improved support coverage, you need to upgrade to GitHub Team or GitHub Enterprise Cloud)?”
- Additional features include:
 - Full feature set for private repositories
 - Organization-level features
 - SAML single sign-on
 - Improved support coverage

SECTION 2 — Billing, Heroku Plans, GitHub Plans, and Questions (con't)?

=== Service Scope Questions ===

- “Outline/describe the set of services you are asking that I provide.”
- “Will you be asking for coding support? If so, what languages?”
- “Confirming that you want a private repository.”

=== Access & Team Questions ===

- “How many people will access the Heroku and GitHub accounts?”
- “Who will access the accounts?”
- “Is there a preferred Heroku Team name or Team owner?”
- “You must add a payment method to create a Heroku Team.”
- Payment method: [REDACTED]

=== Heroku Dyno Types & Pricing ===

- Basic Dyno:
 - \$7/month
 - ~ \$0.01/hour
 - Cost depends on dyno size (vertical scaling: RAM, CPU, compute)

SECTION 2 — Billing, Heroku Plans, GitHub Plans, and Questions (con't)?

=== Service Scope Questions (con't) ===

- “Outline/describe the set of services you are asking that I provide.”
- “Will you be asking for coding support? If so, what languages?”
- “Confirming that you want a private repository.”

=== Access & Team Questions (con't) ===

- “How many people will access the Heroku and GitHub accounts?”
- “Who will access the accounts?”
- “Is there a preferred Heroku Team name or Team owner?”
- “You must add a payment method to create a Heroku Team.”
- Payment method: [REDACTED]

=== Heroku Dyno Types & Pricing (con't) ===

- Standard 2x Dyno:
 - \$50/month
 - ~ \$0.06/hour
 - Higher performance
- Performance-L Dyno:
 - \$500/month
 - ~ \$0.69/hour
 - Highest performance tier

SECTION 3 — Offboarding Checklist (GitHub, Heroku, AWS, Add-ons)?

=== Offboarding Checklist ===

Recommends for a smooth hand-off with minimal impact on the app are the following steps:

GITHUB SETUP

[] Create a GitHub account for automatic repository transfer.

[] Set up a GitHub Organization.

You may choose one of the following:

- Add [EMAIL REDACTED] as an admin to a newly created repository.
- Or send my GitHub username to the team so they can initiate the transfer.

[] Complete code transfer.

SECTION 3 — Offboarding Checklist con't (GitHub, Heroku, AWS, Add-ons)?

----- HEROKU SETUP -----

- [] Create a Heroku account for automatic transfer of the monolith app.
(Heroku is a container-based cloud platform used to deploy, manage, and scale modern apps.)
- [] Set up a Heroku Team account.
- [] Invite [EMAIL REDACTED] to the Heroku Team.
- [] Transfer the app.
 - Or the team can invite consultant as an admin to accept the transfer.

----- HEROKU ADD-ONS (Transfer Automatically) -----

These add-ons will transfer automatically:

- [] Rollbar
- [] Postgres
- [] Memcache Cloud
- [] Mailgun
- [] Papertrail

SECTION 3 — Offboarding Checklist con't (GitHub, Heroku, AWS, Add-ons)?

----- AWS SETUP (S3 STORAGE) -----

- [] Set up an AWS account to use S3 with the appropriate bucket and configuration.
- The dev team has an automatic CloudFormation template to set this up.

Questions to clarify:

1. Do you want a single AWS account created for consulting supporter?
2. Do you want an AWS account with permissions for a specific S3 bucket?
3. How will we access the CloudFormation template?
4. What S3 storage class do you want?

Options:

- S3 Standard
- Intelligent-Tiering
- One-Zone IA
- Others

5. How will the S3 bucket be used?

- Storage?
- Data lake?

SECTION 4 — Heroku Dyno Sizing, and Performance Notes?

=== Heroku Dyno Sizing & Performance ===

Heroku dynos are lightweight containers that run your app processes. Each dyno runs a single command defined in your Procfile.

----- CALCULATING DYNO REQUIREMENTS -----

Example:

- App requires ~100 MB to process each request.
- 100 MB × 10 = 1 second → 10 requests per second without concurrency.
- If running two processes per dyno, capacity becomes 20 requests per second per dyno.

Formula:

Dynos Needed = Throughput (requests/sec) × Response Time (sec)

Dyno Concurrency

Note:

- Real-world performance varies due to inconsistent response times.
- This often leads to under-provisioning or over-provisioning.

Recommendation:

- Use Heroku Autoscaler for automatic scaling.

SECTION 4 — Heroku Dyno Scaling, and Performance Notes (con't)?

----- RECOMMENDED STARTING CONFIGURATION -----

- Start with ****Standard 2x dynos****, running two app processes per dyno.
- If the 1 GB memory quota is insufficient, upgrade to ****Performance-L**** dynos.

----- HEROKU ACCOUNT CREATION (SANITIZED) -----

Account details:

- Email: [EMAIL REDACTED]
- Password: [PASSWORD REDACTED]
- Username: [ACCOUNT INFO REDACTED]

SECTION 4 — Heroku Dyno Scaling, and Performance Notes (con't)?

----- RECOMMENDED STARTING CONFIGURATION -----

- Start with ****Standard 2x dynos****, running two app processes per dyno.
- If the 1 GB memory quota is insufficient, upgrade to ****Performance-L**** dynos.

----- HEROKU PLATFORM NOTES -----

- You can deploy your app with a few clicks.
- When you create a new app, Heroku assigns a random natural-theme name (e.g., “calm-springs-3345” or “desolate-cliffs-1221”).
- You must use Git to deploy to Heroku.
Example command:
`git push heroku master`
- NOTE: GitHub also supports a “Deploy to Heroku” button for one-click deployment.

SECTION 4 — Heroku Dyno Scaling, and Performance Notes (con't)?

----- HEROKU TOOLBELT / CLI

To deploy from your desktop:

- Install the Heroku CLI.
- Login:
 `heroku login`
- Clone a sample app:
 `git clone https://github.com/heroku/python-getting-started.git`
 `cd python-getting-started`

----- PROCFILE

A Procfile defines the command Heroku runs to start your app.

Example:

`web: gunicorn getting_started.wsgi`

SECTION 4 — Heroku Dyno Scaling, and Performance Notes (con't)?

----- DEPLOYMENT WORKFLOW -----

1. Create the Heroku app:
heroku create
(This creates a Git remote and assigns a random app name.)
2. Deploy code:
git push heroku main
3. Scale dynos:
heroku ps:scale web=1
4. View logs:
heroku logs --tail
5. Check dyno status:
heroku ps

SECTION 4 — Heroku Dyno Scaling, and Performance Notes (con't)?

----- LOCAL DEVELOPMENT -----

- Install dependencies locally.
- Run the app locally.
- Push local changes to Heroku.
- Start a console session:
 `heroku run bash`

----- ADD-ONS -----

Add-ons provide third-party services such as logging, monitoring, and persistence.

Example:

```
heroku addons:create papertrail
heroku addons
```

----- ONE-OFF DYNO -----

Run a one-off dyno for debugging:
 `heroku run bash`

SECTION 5 — GitHub Account Setup & Local Repository Instructions?

=== GitHub Account Creation ===

Account details (sanitized):

- Username: [ACCOUNT INFO REDACTED]
- Email: [EMAIL REDACTED]
- Password: [PASSWORD REDACTED]

Purpose:

- Create a place for your project repository to live online.

ONLINE REPOSITORY SETUP

Steps:

1. Create a new GitHub repository.
2. Repository name example:
 examplepub-repo
3. Visibility:
 Public (anyone can see it)
 or
 Private (restricted access)

SECTION 5 — GitHub Account Setup & Local Repository Instructions (con't)?

=== GitHub Account Creation ===

LOCAL REPOSITORY SETUP

To create a matching local repository on your computer:

1. Open PowerShell.
2. Create a directory:
 `mkdir ~/examplepub-repo`
3. Move into the directory:
 `cd ~/examplepub-repo`
4. Initialize it as a Git repository:
 `git init`

This creates a local Git repository that mirrors the online GitHub repository.

SECTION 5 — GitHub Account Setup & Local Repository Instructions (con't)?

=== GitHub Account Creation ===

NOTES ABOUT GITHUB

- GitHub is a graphical interface built on top of the Git version control system.
- It allows:
 - Version tracking
 - Collaboration
 - Branching and merging
 - Pull requests
 - Issue tracking
- GitHub integrates with Heroku for one-click deployment via the “Deploy to Heroku” button.